

The core challenge: what distinguishes these marks?

Let me think through the geometry/pixel features of each mark type:

Mark	Fill ratio	Circularity	Symmetry	Stroke pattern
Full circle (correct)	High (0.7–1.0)	High	High	Uniform
Partial/sloppy fill	Medium (0.4–0.7)	Medium	Medium	Uneven
Half circle	~0.4–0.5	Low	Low (one axis)	Edge-heavy one side
Concentric circle (unfilled bubble outline)	Very low (0.05–0.15)	High	High	Perimeter only
Tick/checkmark	Low, elongated	Very low	Asymmetric	Angular strokes
Cross/X	Low, elongated	Very low	Diagonal strokes	Two intersecting lines

Key thresholds I'd recommend testing:

- **Fill ratio** (filled pixels / bubble area): >0.45 = candidate answer
- **Circularity** ($4\pi \cdot \text{area} / \text{perimeter}^2$): >0.6 = circular enough
- **Aspect ratio** of filled region: $0.6\text{--}1.4$ = not a tick/cross
- **Edge density** (perimeter pixels / total): concentric circles have very high edge density relative to fill

Now let me build the synthetic OMR filler first so you have test data:## How to use this for your actual scanner

The 5 features and why they matter:

Fill ratio (filled pixels $\div$ bubble area) — the primary gate. Filled circles hit $0.7\text{--}0.95$. Ticks/crosses are $0.10\text{--}0.25$. Concentrics are $0.05\text{--}0.15$. Recommended minimum: **0.42**.

Circularity ($4\pi \cdot \text{area} / \text{perimeter}^2$) — distinguishes blobs from strokes. A perfect circle = 1.0. Ticks score $0.15\text{--}0.35$. Half-circles ~ 0.4 . Recommend minimum: **0.55**.

Aspect ratio — ticks and crosses are elongated (ratio > 2.0). Circles stay near 1.0. Cap at **1.8**.

Edge density (perimeter pixels $\div$ filled pixels) — concentrics have very high edge density because they're all outline and no fill. Cap at **0.55**.

Symmetry (left/right + top/bottom balance) — half-circles fail vertical symmetry badly (~ 0.45). Ticks fail horizontal symmetry. Minimum: **0.60**.

For your OpenCV pipeline, the decision logic should be:

```
def is_valid_mark(fill_ratio, circularity, aspect_ratio, edge_density, symmetry):
```

```
if fill_ratio < 0.42: return "EMPTY"  
if aspect_ratio > 1.8: return "TICK_OR_CROSS"  
if edge_density > 0.55 and fill_ratio < 0.20: return "CONCENTRIC"  
if circularity < 0.55 or symmetry < 0.60: return "PARTIAL_REJECT"  
if fill_ratio < 0.54: return "BORDERLINE" # human review zone  
return "VALID"
```

The "borderline" zone (0.42–0.54 fill) is your tolerance band for sloppy but genuine answers. Want me to write the full OpenCV bubble detection + scoring code next?